Oto kompleksowe opracowanie i ściągawka poświęcona **funkcjom w Bashu oraz zastosowaniu polecenia let i obliczeniom arytmetycznym**, przygotowana w czytelnej formie edukacyjnej:

Ściągawka: Funkcje w Bashu i obliczenia arytmetyczne (let)

1. Definiowanie i wywoływanie funkcji

Funkcje pozwalają na grupowanie bloków kodu w celu wielokrotnego użytku.

- **Przekazywanie parametrów:** Do funkcji przekazuje się argumenty dokładnie tak samo jak do zwykłych skryptów czy poleceń (np. myfunction "arg1" "arg2").
- Wewnątrz funkcji parametry są dostępne poprzez zmienne pozycyjne ($1, $2 itd.), a ich łączną liczbę zwraca zmienna specjalna $#.
- Instrukcja return pozwala na przerwanie działania funkcji (często stosowana w walidacji liczby argumentów).

Przykład funkcji z obsługą argumentów:

Bash

```
#!/bin/bash
myfunction() {
        if [ "$#" != 2 ]; then
                return
        fi
        echo "Liczba argumentów: $#"
        for i in "$@"; do
                echo "$i"
        done
}
# Wywołanie i przypisanie wyniku do zmiennej
VAR=$(myfunction "parametr 1" "parametr 2")
echo "$VAR"
```[cite: 77]

---

2. Polecenie `let` oraz operacje arytmetyczne

Polecenie `let` umożliwia przeprowadzanie obliczeń arytmetycznych na liczbach całkowitych (stałoprzecinkowych) bez sprawdzania przepełnienia[cite: 77, 78]. Dzielenie przez zero jest przez interpreter przechwytywane jako błąd[cite: 78].

* **Składnia:** `let arg [arg...]`[cite: 77]
* **Ważna uwaga:** Przy przypisywaniu wartości do zmiennych w poleceniu `let` **nie wolno dawać spacji** przed i za znakiem równości[cite: 78].

Przykład użycia polecenia `let`:

```bash
#!/bin/bash
```

```
liczba1=5
liczba2=6
let wynik=liczba1*liczba2
echo "${wynik}"  # Wynik: 30
```[cite: 78]

---

3. Priorytet operatorów w wyrażeniach arytmetycznych
W obliczeniach Bash obsługuje standardowe operatory (uporządkowane od najwyższego priorytetu)[cite: 78]:
* **Inkrementacja / Dekrementacja:** `id++`, `id--`, `++id`, `--id`[cite: 78]
* **Jednoargumentowe:** `-`, `+`, `!`, `~`[cite: 78]
* **Potęgowanie:** `**`[cite: 78, 79]
* **Arytmetyczne:** `*`, `/`, `%` (mnożenie, dzielenie, reszta) oraz `+`, `-` (dodawanie, odejmowanie)[cite: 78]
* **Przesunięcia bitowe i porównania:** `<<`, `>>`, `<`, `<=`, `>`, `>=`, `==`, `!=`[cite: 78]
* **Operacje bitowe i logiczne:** `&`, `^`, `|`, `&&`, `||`[cite: 78]
* **Warunkowe i przypisanie:** `expr ? expr : expr` oraz `=` i operatory skrócone (`+=`, `*=`, itp.)[cite: 78]

---

4. Obsługa liczb ułamkowych i potęgowania
Ponieważ domyślne operacje arytmetyczne w Bashu działają wyłącznie na **liczbach całkowitych** (np. dzielenie `2 / 5` zwróci `0` zamiast `0.4`), do obliczeń zmiennoprzecinkowych z udziałem ułamków stosuje się zewnętrzne narzędzie `bc` wraz z parametrem `scale`[cite: 79]:

```bash
B=2
C=5
A=$((B / C))
echo $A  # Wynik: 0 (działanie na liczbach całkowitych)[cite: 78, 79]

# Użycie polecenia bc dla precyzji ułamkowej:
echo "scale=5; ($B / $C)" | bc     # Wynik: .40000[cite: 79]
echo "scale=15; ($B / $C)" | bc    # Wynik z wyższą precyzją[cite: 79]

# Potęgowanie:
A=$((B ** 2))
echo $A  # Wynik: 4[cite: 79]
```[cite: 78, 79]